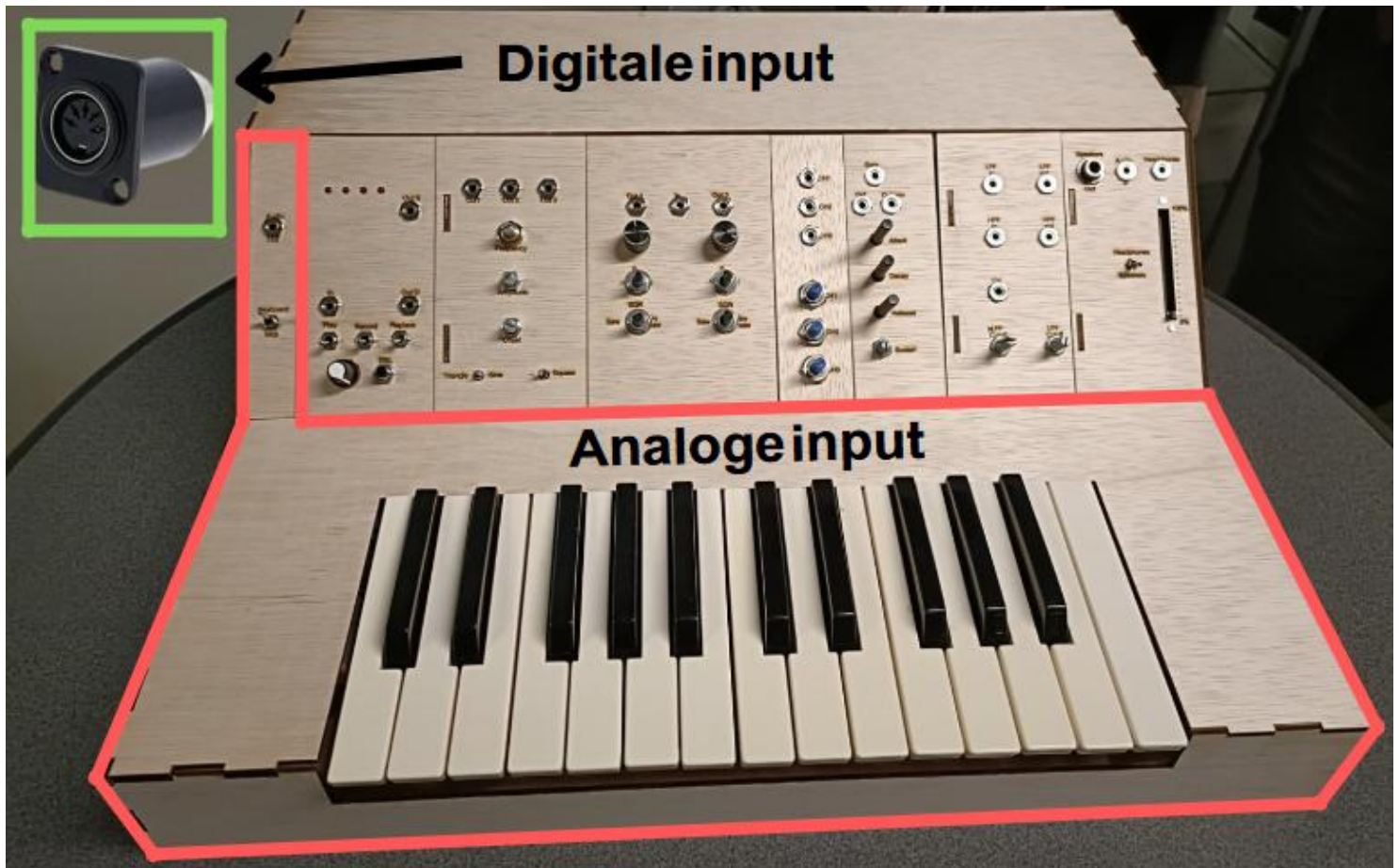


Keyboard inputs voor de synthesizer

Zonder input weinig output



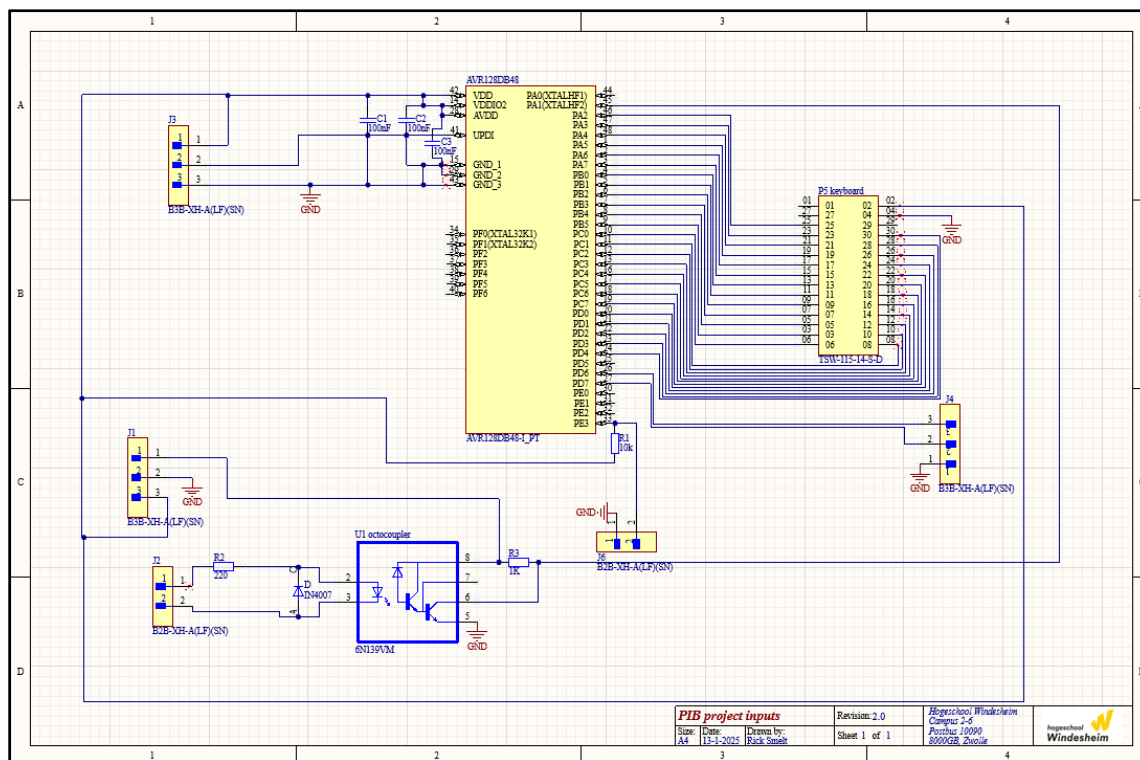
De input van een synthesizer is onmisbaar, net zoals een stuur voor een auto. Zonder stuur kun je niet sturen, hoe krachtig de motor ook is. Hetzelfde geldt voor een synthesizer: zonder input, zoals een keyboard, blijft het apparaat wel aanwezig, maar kun je er niets mee doen. Het keyboard is de directe schakel tussen de muzikant en de mogelijkheden van de synthesizer.

Net zoals een stuur de richting van een auto bepaalt, geeft het keyboard de muzikant controle over wat er door de synthesizer wordt gegenereerd. Met elke toets die wordt ingedrukt,

activeren specifieke tonen, ritmes en geluiden. Het keyboard biedt de muzikant de mogelijkheid om precies te bepalen welke muziek er wordt gecreëerd, of het nu gaat om eenvoudige melodieën of complexere composities.

Zonder deze input blijft de synthesizer beperkt tot zijn technische mogelijkheden, zonder dat er daadwerkelijke muziek uit voortkomt. Het keyboard maakt de synthesizer een veelzijdig instrument dat de creativiteit van de muzikant kan omzetten in geluid, en zorgt ervoor dat het volledige potentieel van het apparaat benut wordt.





Figuur 1

Dit schema laat alle verschillende aansluitingen zien.

Volledige Schakeling

De gehele schakeling is schematisch weergegeven in **Figuur 1**. Voor de verwerking van binnenkomende inputs via de keyboards wordt gebruikgemaakt van de AVR128DB48 microcontroller. Deze controller wordt gebruikt voor het aansturen van de Digital-to-Analog Converter (DAC) ook heeft deze controller het aantal GPIO-pins die nodig zijn om alle toetsen individueel te kunnen controleren. Er is voor gekozen om deze microcontroller te programmeren in MPLAB X IDE van Microchip Technology, en dit programma wordt via de UPDI-pin in poort J3 geüpload naar de controller. Vanuit de voedingsconnector J1 worden 3.3 volt en 5 volt ontvangen. De 3,3V-voeding wordt via condensatoren C1 ... C3 gekoppeld aan de microcontroller voor voedingsontkoppeling, zodat een stabiele werking van de microcontroller wordt gegarandeerd.

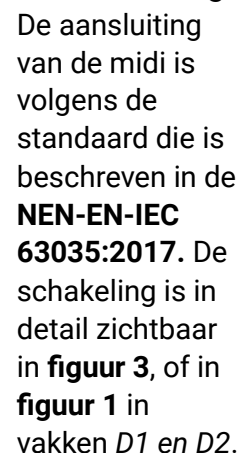


Figuur 2

Afbeelding van het keyboard PCB met daarop zichtbaar de drukknoppen

Analoge schakeling

Het analoge keyboard is aangesloten op de 3,3 volt, dit zodat er een duidelijk 3,3 volt signaal wordt verstuurd richting de microcontroller om aan te geven of er een toets is ingedrukt of niet. De inputs van dit keyboard komen binnen via de pin-header die is geplaatst op het PCB dit is P5 in het schema. Via de 30 pins header wordt 3.3v en gnd ook meegestuurd naar de toetsen. Op het PCB waar de toetsen op zijn gemonteerd zitten 50 drukschakelaars (deels zichtbaar in **figuur 2**). Elke toets heeft er 2 voor redundantie. Deze drukschakelaars zijn normaal open, en hebben dus een pull-downweerstand nodig. Omdat de drukschakelaars zelf al een grote weerstand hadden (200Ω) was een 1KΩ pull-downweerstand niet voldoende en is deze verhoogd naar 100KΩ zodat als de toets niet is ingedrukt dat er dan 0 wordt terug gestuurd naar de microcontroller. Als deze wel wordt ingedrukt laat deze de 3.3V door omdat 200Ω veel kleiner is dan 100KΩ.



Figuur 3

De optocoupler is een belangrijk onderdeel van deze schakeling. Het biedt galvanische scheiding tussen het MIDI-keyboard en de microcontroller, waardoor ongewenste stromen of spanningsverschillen worden voorkomen. Dit zorgt voor een betrouwbare en veilige werking van de schakeling.

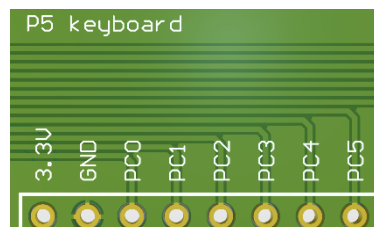
Vervolgens is er nog een switch aangesloten op de connector *J6*. Aan deze switch is een pull-upweerstand toegevoegd, zodat het signaal standaard 3,3 volt is op de pin van de microcontroller. Wanneer de switch gesloten wordt, zal de pin worden verbonden met de GND en daarmee laag worden.



Interface paneel van de Inputs op de synthesizer met een 3.5 mm audio jack en een switch voor keuze tussen analoog en digitaal

PCB-design

Het PCB-design (**Figuur 5**) is ontworpen in Altium Designer. Bij het ontwerp is rekening gehouden met de plaatsing van diverse componenten. De drie condensatoren zijn zo dicht mogelijk bij de microcontroller geplaatst om optimale voedingsontkoppeling te garanderen. De verschillende connectoren zijn zoveel mogelijk aan de rand van de printplaat geplaatst om een goede toegankelijkheid bij het aansluiten te waarborgen. De grote pinheader aan de rechterkant is strategisch gepositioneerd om de sporen naar en van de microcontroller zo kort mogelijk te houden. Ook is er op deze manier geen gebruik gemaakt van via's in de lijnen naar deze header omdat alle lijnen nu parallel naast elkaar kunnen lopen, zie **figuur 6**. Ook zijn de lijnen voor de digitale decoder extra groot gemaakt zodat er geen bottleneck zou zitten in de maximale stromen die door de lijnen zouden

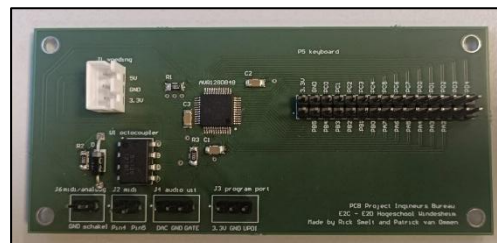


Figuur 6

Afbeelding van vergrote lijnen op het PCB

kunnen gaan. Er zijn op dit PCB ook 4 montage gaten gemaakt. Alle

gaten zijn het standaard m3 formaat waardoor montage in de behuizing gemakkelijk verloopt. Voor deze componenten waren geen verdere koollichamen nodig. Voor de benodigde componenten bekijk de onderdelenlijst hiernaast:



Figuur 5

Afbeelding van zelfontworpen PCB

Onderdelenlijst:

Weerstanden:

SMD 1206	Throughhole
R1 = 100k Ω	25x 100k Ω
R2 = 220 Ω	
R3 = 1k Ω	

Condensatoren:

C1, C2 en C3 = 100nF

Diodes:

D = 1n4007

Optocoupler:

U1 = 6n139

Microcontroller:

AVR128DB48

Connectoren:

- P5 = Pin-header 2 x 15
- J2, J6 = Pin-header 2 x 2
- J3, J5 = Pin-header 2 x 3
- J1 = JST-connector 3 pins

Overige onderdelen:

- Switch
- 25 knoppen/toetsen

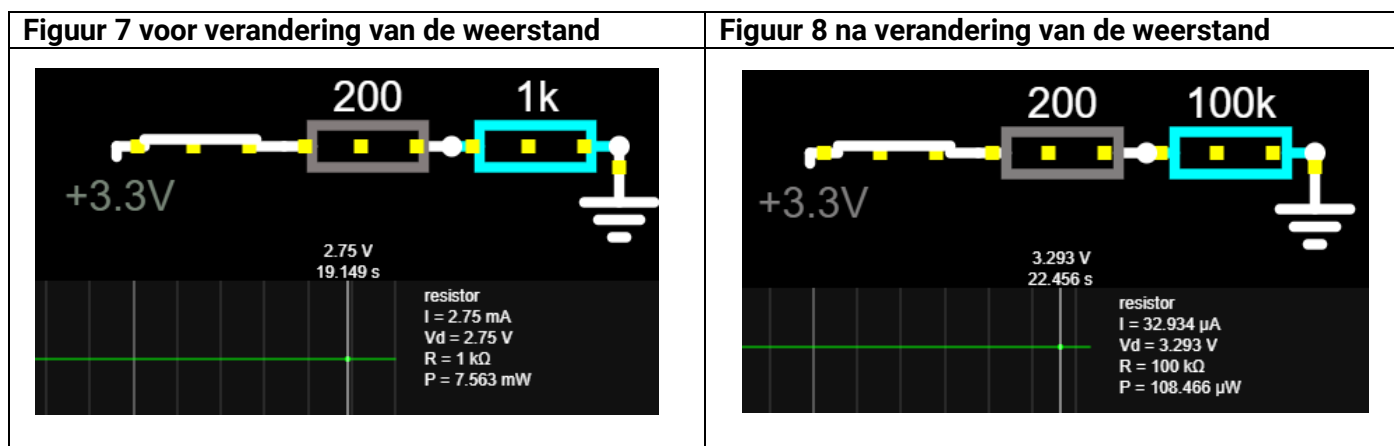
Testen

Tijdens de testfase van dit project werd gebruik gemaakt van de AVR128DB48 Curiosity Nano Evolution Kit. Deze kit bevatte onder andere de AVR128DB48-microcontroller, een onboard debugger en twee 24-pins headers voor aansluiting op een breadboard. Aangezien in dit project gebruik werd gemaakt van een hergebruikt toetsenbord, waren er al toetsen en een PCB beschikbaar.

Problemen en oplossingen tijdens de testfase

1. Weerstandsprobleem

Aan het begin van de testfase werden drie toetsen aangesloten, om te controleren of de signalen correct door de microcontroller werden ontvangen. Het bleek echter dat de drukknoppen op het PCB een aanzienlijke interne weerstand hadden, namelijk circa 200 Ω . Eerder waren er op het PCB 1 k Ω -weerstanden gesoldeerd, wat resulteerde in een te lage spanningswaarde naar de microcontroller. De binnenkomende spanning was ongeveer 2,75 V in plaats van de verwachte 3,3 V. Om dit probleem op te lossen werden de 1 k Ω -weerstanden vervangen door 100 k Ω -weerstanden. Hierdoor werd de binnenkomende spanning hersteld naar 3,293 V, zoals weergegeven in **figuur 7** (voor de wijziging) en **figuur 8** (na de wijziging).



2. Kortsluitingsprobleem

Tijdens het aansluiten van alle 25 toetsen op de Curiosity Nano werd een ander probleem geconstateerd. Hoewel de DAC-waarden correct waren ingesteld in MPLAB X IDE en de juiste DAC-voltages zonder toetsen werden gemeten met een multimeter, traden afwijkingen op zodra het PCB met de drukknoppen op het metalen frame van het toetsenbord werd bevestigd. De DAC-voltages waren incorrect, en er waren momenten waarop meer toetsen niet functioneerden dan wel. Na grondige inspectie werd vastgesteld dat het PCB kortsluiting maakte met het metalen frame. Om dit probleem op te lossen, werd het volledige PCB geïsoleerd door het in elektriciteitstape te wikkelen voordat het opnieuw werd gemonteerd. Na deze aanpassing werden de juiste voltages weer gemeten, en functioneerde het toetsenbord naar behoren.

Aanbeveling

Bij het testen van een soortgelijke opstelling is het aan te raden om in een vroeg stadium te controleren op mogelijke kortsluiting tussen het PCB en metalen componenten. Dit kan veel tijd en moeite besparen tijdens de foutopsporing. Meet of controleer voortijdig ook altijd even de interne weerstand van de drukknoppen, zodat je niet de verkeerde weerstanden aansluit.

Software ontwerp

Aan de hand van de flowchart (**Figuur 9**) wordt de werking van de code stap voor stap uitgelegd. Wanneer het systeem wordt gestart, wordt als eerste bepaald in welke modus het systeem moet functioneren: de analoge inputmodus of de digitale inputmodus. Deze keuze wordt gemaakt met behulp van een eenvoudig IF-statement. (**Figuur 10**)

```
int main(void) {
    SYSTEM_Initialize();

    if (schakelaar_GetValue()) {
        Analoge_Input();
    }
    else {
        Digitale_Input();
    }
}
```

Figuur 10

Statement om de modus te checken

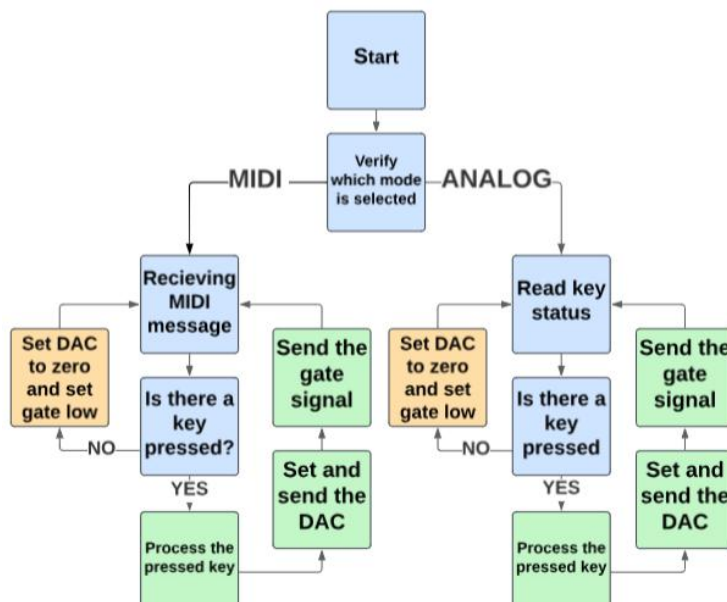
Wat hier gebeurt is vrij simpel. In het IF-statement wordt gecontroleerd of de schakelaar een waarde van **1** heeft (hoog). Als dit het geval is, zal het systeem de functie **Analoge_Input()** uitvoeren. Wanneer de schakelaar een waarde van **0** heeft (laag), wordt de functie **Digitale_Input()** uitgevoerd.

Functies en states aanmaken

Voordat er gekeken wordt naar de functies voor de inputs moeten er eerst nog een aantal dingen worden aangemaakt. De `uint16_t DAC_value` is een globale variabele die de waarde van de DAC op gaat slaan, deze is standaard 0. Deze variabele wordt gebruikt om de huidige uitgangswaarde van de DAC te beheren.

De functie `DAC_SetOutput` accepteert een 16-bits ongetekende integer (`uint16_t value`) en schrijft deze naar de DATA-register van DAC0. Hierdoor kunnen er verschillende analoge waardes verstuurd worden naar de uitgang.

De `gate_pulse`-functie is verantwoordelijk voor het genereren van een korte puls. Deze functie zet de poort kort op 0 V, wacht 10 ms, en zet de poort daarna weer op 3,3 V. Deze functionaliteit is belangrijk voor de ADSR-module, omdat deze hierdoor gemakkelijker kan detecteren of er een nieuwe toets is ingedrukt.



Figuur 9

Flowchart van het softwaresysteem

```
uint16_t DAC_value = 0;
void DAC_SetOutput(uint16_t value) {
    DAC0.DATA = value;
}

void gate_pulse(void) {
    gate_SetLow();
    _delay_ms(10);
    gate_SetHigh();
}

typedef enum {
    IDLE, BUTTON_1, BUTTON_2, BUTTON_3, BUTTON_4, BUTTON_5,
    BUTTON_6, BUTTON_7, BUTTON_8, BUTTON_9, BUTTON_10,
    BUTTON_11, BUTTON_12, BUTTON_13, BUTTON_14, BUTTON_15,
    BUTTON_16, BUTTON_17, BUTTON_18, BUTTON_19, BUTTON_20,
    BUTTON_21, BUTTON_22, BUTTON_23, BUTTON_24, BUTTON_25
} ButtonState;
```

Figuur 11

Functies en aanmaken van buttonStates.

Ten slotte is er de **ButtonState**-enumeratie, die 26 verschillende states definieert (dit kan worden uitgebreid op basis van het aantal toetsen). Iedere **BUTTON** verwijst naar een aparte toets op het keyboard. De **IDLE**-state is voor het geval dat geen van de toetsen is ingedrukt.



Analoge input

Voor de analoge input is er een aparte functie aangemaakt waar in verschillende dingen gebeuren. Dit is de functie `Analoge_Input()`.

Allereerst wordt de DAC-uitgang op 0 gezet met de functie `DAC_SetOutput(0)`. Daarnaast worden twee variabelen, `currentState` en `lastState`, geïnitialiseerd op `IDLE`. Deze stappen worden gezet zodat we elke keer weer op dezelfde manier starten.

De code zal dan in de While loop gaan. Nu wordt er constant gecheckt of er een toets wordt ingedrukt. Dit gebeurt met behulp van een serie if- en else if-condities. Elke toets heeft een eigen controlefunctie, zoals `OCT2_C_GetValue()`. Als een bepaalde toets wordt gedetecteerd, wordt de variabele `currentState` ingesteld op de bijbehorende state, bijvoorbeeld bij `OCT3_C` hoort `BUTTON_25`. Als geen enkele toets wordt ingedrukt, wordt `currentState` op `IDLE` gezet.

Als de huidige state anders is dan de vorige state betekent dit dat er een wijziging heeft plaatsgevonden, zoals het indrukken van een nieuwe toets of het loslaten van een toets.

Als er een toets is ingedrukt, wordt de index van de toets berekend door de waarde van `currentState` te reduceren met `BUTTON_1`. Deze stap is nodig, omdat de waarde van `BUTTON_1` al 311 is. Zonder deze correctie zou je een onjuiste, te hoge waarde krijgen. Deze index wordt vervolgens gebruikt om een analoge waarde te berekenen volgens de formule: $DAC_value = 311 +$

```
void Analoge_Input() {
    DAC_SetOutput(0);
    ButtonState currentState = IDLE;
    ButtonState lastState = IDLE;
    while (1) {
        // Detecteer welke knop wordt ingedrukt.
        if (OCT3_C_GetValue()) currentState = BUTTON_25;
        else if (OCT3_B_GetValue()) currentState = BUTTON_24;
        else if (OCT3_Bb_GetValue()) currentState = BUTTON_23;
        else if (OCT3_A_GetValue()) currentState = BUTTON_22;
        else if (OCT3_Ab_GetValue()) currentState = BUTTON_21;
        else if (OCT3_G_GetValue()) currentState = BUTTON_20;
        else if (OCT3_Gb_GetValue()) currentState = BUTTON_19;
        else if (OCT3_F_GetValue()) currentState = BUTTON_18;
        else if (OCT3_Fb_GetValue()) currentState = BUTTON_17;
        else if (OCT3_Eb_GetValue()) currentState = BUTTON_16;
        else if (OCT3_D_GetValue()) currentState = BUTTON_15;
        else if (OCT3_Db_GetValue()) currentState = BUTTON_14;
        else if (OCT3_C_GetValue()) currentState = BUTTON_13;
        else if (OCT3_Bb_GetValue()) currentState = BUTTON_12;
        else if (OCT3_Bb_GetValue()) currentState = BUTTON_11;
        else if (OCT3_A_GetValue()) currentState = BUTTON_10;
        else if (OCT3_Ab_GetValue()) currentState = BUTTON_9;
        else if (OCT3_G_GetValue()) currentState = BUTTON_8;
        else if (OCT3_Gb_GetValue()) currentState = BUTTON_7;
        else if (OCT3_F_GetValue()) currentState = BUTTON_6;
        else if (OCT3_Fb_GetValue()) currentState = BUTTON_5;
        else if (OCT3_Eb_GetValue()) currentState = BUTTON_4;
        else if (OCT3_D_GetValue()) currentState = BUTTON_3;
        else if (OCT3_Db_GetValue()) currentState = BUTTON_2;
        else if (OCT3_C_GetValue()) currentState = BUTTON_1;
        else currentState = IDLE;

        if (currentState != lastState) {
            if (currentState != IDLE) {
                uint8_t buttonIndex = currentState - BUTTON_1; // Bereken index van de knop.
                DAC_value = 308 + buttonIndex * 25.6; // Bereken de DAC-uitgang voor de toets.
                DAC_SetOutput(DAC_value << 6); // Zet de DAC-uitgang.
                gate_pulse(); // Geef een puls op de gate.
            } else {
                DAC_value = 0;
                DAC_SetOutput(DAC_value << 6);
                gate_SetLow();
            }
            lastState = currentState;
        }
    }
}
```

Figuur 12

Functie voor de analoge input (ingebouwd keyboard)

`buttonIndex * 25,85`. Deze formule hoort bij een V_{ref} van 3.3 volt.

Met deze waarde wordt de DAC ingesteld. Daarnaast wordt een korte puls gegenereerd met de functie `gate_pulse`.

Als geen enkele toets wordt ingedrukt wordt de DAC-uitgang op 0 gezet, en wordt de gate laag gezet met de functie `gate_SetLow`. Tot slot schrijven we de `currentState` na de `LastState`.

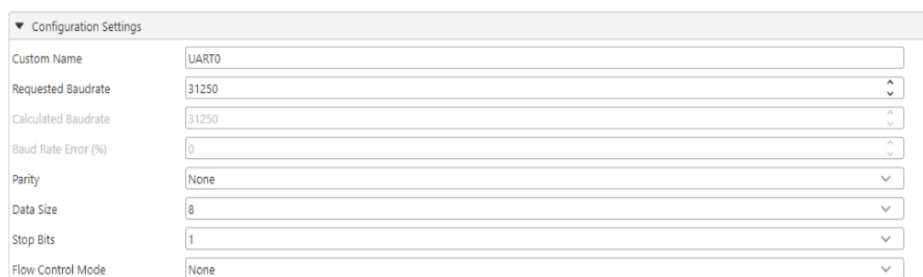


Digitale input

Helaas is er nog geen oplossing gevonden om het MIDI-signaal te verwerken. Er zijn echter al een aantal belangrijke instellingen gedaan, en het is bekend hoe het signaal eruitziet.

De belangrijkste instellingen voor het uitlezen van het MIDI-signaal via UART worden weergegeven in

Figuur 13. De standaard baudrate voor MIDI-berichten is 31.250 bits per seconde. Dit betekent dat de baudrate in de UART-instellingen hierop moet worden ingesteld. In het programma MPLAB X IDE kan dit eenvoudig worden geconfigureerd via de MCC. Naast de baudrate zijn ook de datagrootte (data size) en het aantal stopbits belangrijke instellingen: Data size: 8 bits en Aantal stopbits: 1. Deze instellingen zijn essentieel om MIDI-berichten correct te ontvangen en te verwerken.



Configuration Settings	
Custom Name	UART0
Requested Baudrate	31250
Calculated Baudrate	31250
Baud Rate Error (%)	0
Parity	None
Data Size	8
Stop Bits	1
Flow Control Mode	None

Figuur 13

Instellingen UART voor het uitlezen van Midi-berichten

Het signaal wat binnenkomt bij het gebruikte keyboard was 3 bytes. De eerste byte is het kanaal waar het keyboard op aangesloten is. Deze is voor de verwerking van het midi signaal in dit geval niet belangrijk. Byte 2 daarin tegen is zeer belangrijk, dit is namelijk de toon die ingedrukt wordt en staat daarmee direct in verband met de toonhoogte. De derde byte heeft informatie over de aanslaggevoeligheid, dit kan worden gebruikt om de Gate aan te sturen.



Eindresultaat

Het eindresultaat van het gehele project is zichtbaar in **figuur 14** hieronder. Het beschreven onderdeel van de synthesizer is zichtbaar in **figuur 15**.



Figuur 14

De gehele synthesizer met alles modules.



Figuur 15

Omlijnning van de eerder beschreven module



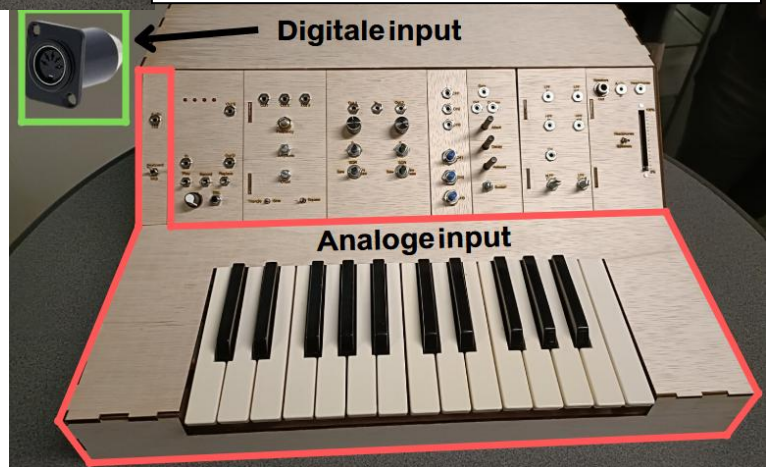
Op het moment van schrijven is er helaas nog geen muziek uit de synthesizer gekomen, dit komt voornamelijk door het defect van de VCO-module, wel kan er worden gezegd dat de analoge input correct werkt op zichzelf. Voor verbinding met de andere modules is er sterk aan te raden dat er nog een eind-buffer nodig is voor de gate en toonhoogte, dit kan worden gedaan met een op-AMP. Deze ontbreekt nog in het ontwerp, omdat dit probleem pas op de presentatiedatum werd ontdekt. De eindbuffer zorgt namelijk voor een lage uitgangsimpedantie, waardoor variaties in ingangs impedantie van andere modules geen invloed hebben op het signaal.

Belangrijke info voor het aansluiten

Als wordt besloten om het ontwerp te kopiëren, is het goed om te weten dat de tekst op de PCB wat betreft de DAC en de gate is omgedraaid. De pin waar "gate" onder staat is in werkelijkheid de DAC, en andersom: waar "DAC" onder staat, is de gate.

Aanbevelingen

De referentiespanning (V_{ref}) is momenteel ingesteld op 3,3 volt voor de DAC. Het wordt aanbevolen om deze op 3 volt te zetten. Dit maakt het eenvoudiger om de waarde vanaf 1023 te laten beginnen, in plaats van een tussenliggende waarde.



Voor het controleren van ingedrukte toetsen wordt hier het IF-statement gebruikt. In dit geval werkt dit goed genoeg, maar het is niet ideaal. Eigenlijk wil je dat wanneer je een tweede toets indrukt zonder de vorige toets los te laten, de nieuwe toets de vorige vervangt. Met deze methode werkt het echter maar één kant op, in dit geval alleen naar rechts, terwijl je liever wilt dat dit in beide richtingen werkt.

Communicatie

Rick Smelt

Rick.Smelt@windesheim.nl

Patrick van Ommen

Patrick.van.Ommen@windesheim.nl

Onder begeleiding van

Richard Rosing: r.rosing@windesheim.nl

Marco Winkelman: m.winkelman@windesheim.nl



Pulse Soundworks

University of
Applied Sciences

Windesheim

